Université Abdelmalek Essaâdi

École Nationale des Sciences Appliquées

Al-Hoceima

Filière : Transformation Digitale et Intelligence Artificielle 2

Rapport TP 2

Applications web avancées avec Java et Spring

Création d'une application de gestion des clients et des réservations

Réalisé par :

Moaad Daoudi

Encadré par :

Pr. Mohamed Cherradi

Année Universitaire 2025/2026

Table des matières

Introduction **3**

1 Modélisation et Architecture globale **4**
1.1 Architecture de l'Application 4
1.2 Modèles de Données (Models) 4
1.2.1 Classe `Client.java` 5
1.2.2 Classe `Reservation.java` 5

2 Développement des Servlets et Vues (JSP) **7**
2.1 Couche Contrôleur (Servlets) 7
2.1.1 Servlet `creerClient` 7
2.1.2 Servlet `creerReservation` 8
2.2 Couche Vue (Pages JSP & HTML) 8
2.2.1 Formulaire d'inscription (`inscription.jsp`) 8
2.2.2 Page de confirmation (`infoClient.jsp`) 9
2.2.3 Formulaire de Réservation (`reservation.jsp`) 10
2.2.4 Confirmation de Réservation (`infoReservation.jsp`) . 10

Conclusion **12**

Table des figures

1.1 Diagramme de l'architecture MVC (Vues et Servlets) du projet TP2 4
1.2 Code source de la classe Client 5
1.3 Code source de la classe Reservation 6

2.1 Méthode doPost du Servlet `creerClient` 8
2.2 Servlet `creerReservation` 8
2.3 Interface utilisateur du formulaire d'inscription 9
2.4 Rendu final : Récapitulatif des informations du client 9
2.5 Interface utilisateur du formulaire de réservation 10
2.6 Rendu final : Récapitulatif de la réservation 11

Introduction

Contexte

Dans le cadre de notre formation, ce rapport synthétise le travail pratique réalisé pour le module de Programmation Web JEE. Face à l'évolution des architectures web, la maîtrise des technologies basiques comme les Servlets et les pages JSP (JavaServer Pages) reste primordiale pour comprendre le fonctionnement interne des requêtes et réponses HTTP dans l'écosystème Java.

Objectifs du document

L'objectif principal de ce document est de présenter de manière exhaustive l'implémentation d'une application web simplifiée (architecture MVC) dédiée à la gestion des clients et à la soumission de réservations. Nous détaillerons la modélisation des données, le rôle de chaque contrôleur, ainsi que la conception des vues dynamiques gérant l'interaction avec le navigateur.

Chapitre 1

Modélisation et Architecture globale

1.1 Architecture de l'Application

L'application a été construite pour refléter un design en couches de type Modèle-Vue-Contrôleur (MVC). Le client interagit avec les pages JSP (Vues), qui transmettent les données vers les Servlets correspondants (Contrôleurs). Ceux-ci valident et manipulent les objets métiers (Modèles) avant de retourner un résultat vers une interface utilisateur de récapitulation.

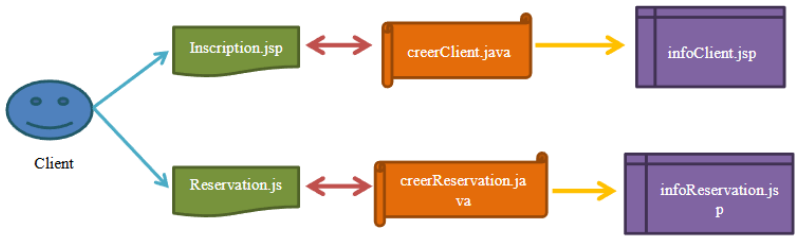

FIGURE 1.1 – Diagramme de l'architecture MVC (Vues et Servlets) du projet TP2

1.2 Modèles de Données (Models)

La couche métier s'organise autour des classes principales servant à stocker l'information durant le cycle de vie de la requête. Ces classes (JavaBeans) respectent le principe d'encapsulation.

1.2.1 Classe `Client.java`

La classe `Client` encapsule les données personnelles des utilisateurs. Elle contient les attributs privés tels que `nom`, `prenom`, `adresse`, `telephone`, et `email`, accompagnés d'un constructeur par défaut et des méthodes accesseurs (Getters) et mutateurs (Setters).

```java
package ma.ensah.tp2.models;

public class Client {
    private String nom;
    private String prenom;
    private String telephone;
    private String email;

    public Client(String nom, String prenom, String telephone, String email) {
        this.nom = nom;
        this.prenom = prenom;
        this.telephone = telephone;
        this.email = email;
    }

    public String getNom() {
        return nom;
    }

    public String getPrenom() {
        return prenom;
    }

    public String getTelephone() {
        return telephone;
    }

    public String getEmail() {
        return email;
    }
}
```

FIGURE 1.2 – Code source de la classe Client

1.2.2 Classe `Reservation.java`

La classe `Reservation` représente la demande de l'utilisateur. Elle stocke les détails concrets comme la `date` de réservation, le `type` de chambre ou service, et génère potentiellement un code de confirmation. Elle sera manipulée en mémoire afin d'être passée aux différentes vues.

```java
package ma.ensah.tp2.models;

public class Reservation {
    private Client client;
    private String type;
    private double prix;
    private String vue;

    public Reservation(Client client, String type, double prix, String vue) {
        this.client = client;
        this.type = type;
        this.prix = prix;
        this.vue = vue;
    }

    public String getType() {
        return type;
    }

    public double getPrix() {
        return prix;
    }

    public String getVue() {
        return vue;
    }
}
```

FIGURE 1.3 – Code source de la classe Reservation

Chapitre 2

Développement des Servlets et Vues (JSP)

2.1 Couche Contrôleur (Servlets)

Le cœur fonctionnel s'articule autour de multiples Servlets qui gèrent le routage des actions. Ils héritent de la classe `HttpServlet` et redéfinissent principalement la méthode `doPost`.

2.1.1 Servlet `creerClient`

Ce contrôleur intercepte la méthode `POST` provenant du formulaire d'inscription.

- **Récupération :** Il récupère les paramètres de la requête web via `request.getParameter("nomChamp")`.
- **Modélisation :** Il instancie un nouvel objet `Client` et lui injecte les données reçues.
- **Transmission :** Il stocke cet objet dans l'objet requête avec `request.setAttribute("cli`
  `objetClient)`.
- **Routage :** Il transfère finalement le flux (`forward`) vers la page de confirmation `infoClient.jsp`.

```java
protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        request.setCharacterEncoding("UTF-8");
        String nom = request.getParameter("nom");
        String prenom = request.getParameter("prenom");
        String telephone = request.getParameter("telephone");
        String email = request.getParameter("email");

        if(nom.isEmpty() || prenom.isEmpty() || telephone.isEmpty() || email.isEmpty()) {
            request.setAttribute("error", "Ooops, erreur !!! Vous devez remplir tous les champs");
            request.getRequestDispatcher("/inscription.jsp").forward(request,response);
        } else {
            Client client = new Client(nom,prenom,telephone,email);
            request.setAttribute("client", client);
            request.getRequestDispatcher("/infoClient.jsp").forward(request, response);
        }
    }
```

FIGURE 2.1 – Méthode doPost du Servlet `creerClient`

2.1.2 Servlet `creerReservation`

De manière similaire, lors d'une tentative de réservation, cette Servlet s'assure de lire les données du formulaire correspondantes, formalise l'objet `Reservation`, et route vers la page `infoReservation.jsp`.

```java
protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        request.setCharacterEncoding("UTF-8");
        String nom = request.getParameter("nom");
        String prenom = request.getParameter("prenom");
        String telephone = request.getParameter("telephone");
        String email = request.getParameter("email");
        String type = request.getParameter("type");
        String prix = request.getParameter("prix");
        String vue = request.getParameter("vue");

        if(nom.isEmpty() || prenom.isEmpty() || telephone.isEmpty() || email.isEmpty() || type.isEmpty() || prix.isEmpty() || vue.isEmpty()) {
            request.setAttribute("error", "Ooops, erreur !!! Vous devez remplir tous les champs");
            request.getRequestDispatcher("/reservation.jsp").forward(request,response);
        } else {
            Client client = new Client(nom,prenom,telephone,email);
            double prixDouble = Double.parseDouble(prix);
            Reservation reservation = new Reservation(client, type, prixDouble, vue);
            request.setAttribute("reservation", reservation);
            request.getRequestDispatcher("/infoReservation.jsp").forward(request, response);
        }
    }
```

FIGURE 2.2 – Servlet `creerReservation`

2.2 Couche Vue (Pages JSP & HTML)

Les vues assurent le contact final avec l'utilisateur.

2.2.1 Formulaire d'inscription (`inscription.jsp`)

C'est la page web qui contient le formulaire HTML (méthode POST, action ciblée vers '/creerClient') permettant la saisie des informations du

client.

FIGURE 2.3 – Interface utilisateur du formulaire d'inscription

2.2.2 Page de confirmation (`infoClient.jsp`)

Il s'agit de l'écran de succès. Il récupère l'objet "client" injecté par le Servlet et utilise soit des scriptlets JSP (ex : `<%= client.getNom() %>`), soit les Expression Language (EL) (ex : `${client.nom}`) pour présenter dynamiquement à l'utilisateur le récapitulatif détaillé des actions.

FIGURE 2.4 – Rendu final : Récapitulatif des informations du client

2.2.3 Formulaire de Réservation (`reservation.jsp`)

La logique de conception reste identique à celle du client. Le formulaire `reservation.jsp` capte les informations relatives à la réservation (date, type, etc.) et les envoie au Servlet.

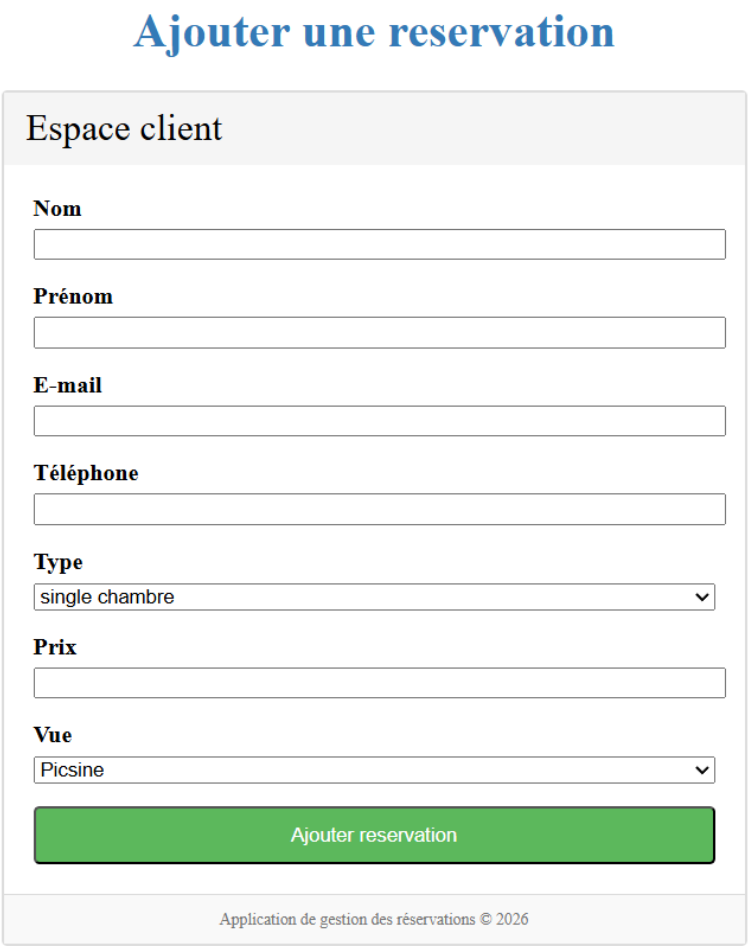

FIGURE 2.5 – Interface utilisateur du formulaire de réservation

2.2.4 Confirmation de Réservation (`infoReservation.jsp`)

Une fois la réservation traitée, la page `infoReservation.jsp` restitue visuellement les données sous forme de récapitulatif de la commande pour confirmer l'opération à l'utilisateur.

FIGURE 2.6 – Rendu final : Récapitulatif de la réservation

Conclusion

La réalisation de ce second TP a permis d'acquérir une compréhension solide de la technologie JEE de base. Connecter des pages JSP à des Servlets manipulant des objets métiers a constitué un prérequis fondamental, préparant le terrain à des solutions d'accès aux données plus complexes comme l'intégration, dans un futur proche, d'une base de données et de frameworks avancés.